

Pre-tool hooks and extending the workspace

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN

Session 6 of 12 · English notes

Pre-tool hooks run *before* Edit, Write, or Bash land. Session 6 used them to exclude protected paths and to deny dangerous commands — especially when permission mode is auto. The same hour showed how to **extend the workspace** beyond one repo's markdown: install marketplace plugins (Playwright and similar), or bundle your own agents, hooks, and skills as a Claude Code plugin (`plugin.json` + marketplace manifest) so other projects inherit the toolkit.

PLUGIN FOLDER NAMES WILL MOVE

The demo used `.claude-plugin / marketplace.json / plugin.json` schemas, user vs project vs local-repo install, and a “To Do Toolkit” package. Syntax errors (wrong source path, circular references, needing a reload) ate a lot of live time. Copy structure from current **Claude Code** docs, not from the recording's trial-and-error.

Pre-tool use: deny before damage

Rajesh's clarification: blocking `rm` is a **pre** hook, not post. After the file is gone, prettier will not help. The trainer's `settings.json` had *multiple matchers* under `pre-tool-use`: one path to `protect-files.sh`, another to `block-dangerous-commands.sh`. Matchers select which tool (Write vs Bash) runs which script.

- Protect-files: skip `.env`, `package-lock.json`, `.git`, ORM secret files (Prisma was the example), anything else you add.
- Dangerous commands: patterns you refuse to let the shell run. Lalit noted that `git restore / revert` can delete content without the token `rm` — extend the denylist, do not assume one binary name is enough.
- Auto mode: “Claude will not ask whether to update a file or execute a command.” Hooks are the remaining brake.
- Sub-agents inherit the same need: unattended reviewers with Write still must not touch secrets.

When the hook “does not fire”

The live `rm test.txt` demo failed until: reload VS Code, reload webviews (Claude is a web component in the IDE), fix the filename, and try from the **terminal** client. A non-blocking hook error status still appeared. A preview feature (“chat is the Claude hook”) had to be enabled so the chat window honoured repo settings. Exam takeaway: configuration is not complete until the same surface you use in anger has executed the matcher. Python/anything-in-shell is allowed; timing is determined by the event, not by the language.

Marketplace plugins as instant workspace extension

Plugins install a set of hooks, skills, and agents into the Claude working space. Scope: user, this project, or local repo. Example: an official Playwright Claude plugin — after install, settings listed it; skills/commands may need a terminal reload to show. Front-end-design plugins similarly inject prompt/skill packs. If a plugin has no skills of its own, you still get whatever agents/hooks it shipped. This is how a team standardises “what hooks should exist” without copying shell files into every greenfield repo by hand.

Rolling your own plugin (To Do Toolkit)

1. Gather existing custom agents, hooks, and skills under a toolkit folder.
2. Add the plugin manifest (name, display name, version, description, keywords, author) following the Claude Code plugin schema.
3. Add a marketplace.json (or the current equivalent) that points at the plugin source path without circular references.
4. Place files under the required dot-folder the docs name that week.
5. In Claude Code: manage plugins → add marketplace or import the folder URL → install at project or user scope.
6. Confirm in settings.json that the plugin is referenced; slash commands such as a toolkit backend skill appear without new files in your app tree.

Mritunjay: a plugin is a library you pass to others; they cannot change it unless they have access (GitHub collaborator/contributor). Installed artefacts live in the sandbox (user vs project). That is “extending the workspace” in the Foundation sense: the empty repo plus a plugin is already an opinionated Claude Code environment.

Why this shows up in exam scenarios

Scenario 2 in the trainer’s later list is literally Claude Code for software development: CLAUDE.md, custom slash commands, plan mode vs direct execution. Pre-tool hooks and plugins are the configuration-layer answers when the scenario adds “auto mode,” “do not commit secrets,” or “roll this out to 50 repos.” Other scenarios (support agent, multi-agent research, CI/CD, structured extraction) still assume you know tool allow/deny — pre-tool hooks are the Claude Code-native version of that idea.

ONE CHAIN TO REMEMBER

Ask/plan while learning → auto only after pre-tool denylists work on your actual client → format with post-tool local commands → distribute the lot as a plugin. CLAUDE.md stays for meaning; hooks stay for mechanics.

Classroom logistics (not product, but asked in-session)

Before Vijay started, facilitation repeated: no screen share of confidential data; UST laptops might still block Pearson VUE — exam on a personal computer or at a test centre. That is delivery constraint, not a Claude Code setting.

Official documentation

- **Claude Code overview**

<https://code.claude.com/docs/en/overview>

Hooks matchers, plugins, marketplaces, user vs project install.

- **MCP**

<https://docs.claude.com/en/docs/agents-and-tools/mcp>

Plugins are not MCP servers; they ship skills/agents/hooks. MCP remains for third-party tools.

- **Guardrails**

<https://docs.claude.com/en/docs/test-and-evaluate/strengthen-guardrails>

Pre-tool deny scripts are guardrails on the coding client.

- **Anthropic Academy (Skilljar)**

<https://anthropic.skilljar.com>

Confirm domain weights and the six production scenarios before exam day.

- **Managed Agents quickstart**

<https://platform.claude.com/docs/en/managed-agents/quickstart>

Workspace extension on the platform side once you leave the IDE plugin model.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.